程式設計（115-1）
期中考

題目設計：孔令傑
國立臺灣大學資訊管理學系

本次考試為手寫考試，考題會以紙本提供給大家，考生只能用筆在官方提供的答案紙上作答。考試是 closed-book 的，考生唯一能帶和查閱的是一張考生自己準備的 A4 紙，這張紙的兩面都可以由考生在考前任意記載任何資訊，但這張紙上不能浮貼其它物體。考生在考試期間不可以跟其他人直接或間接討論題目和答案。考試期間請將手機關靜音或是震動，以免影響其他同學。也請勿使用手機（例如看時間），以免有作弊嫌疑。考試時若有問題，可以舉手問助教；作答時可以不依順序，但要清楚標明題號；可以用任何筆，包含鉛筆，只要清楚就行；筆跡要清晰，如果助教看不懂紙上的文字，就只能扣分。

作答時如果要寫 C++ 程式，以本課程教過的（包含截至目前為止所有作業中正面表列可以使用的）語法為原則。如果使用了本課程沒教過的，但是是正確的語法，會被酌予扣分。

這次的考試時間為 ***2026 年 10 月 27 日 14 點 20 分***至當日 ***17 點 20 分***，共 180 分鐘。本次考試總分共 105 分，超過 100 分者的成績以 100 計。

第一題

（20 分；每題 2 分）對下列各敘述，若正確請寫「T」，錯誤請寫「F」。請勿寫任何說明。

(a) 在 C++ 中，一個 `char` 變數在記憶體中通常占用 1 個位元組（byte）。

(b) 在 C++ 中，運算式 `7 / 2 * 2.0` 的運算結果為 7.0。

(c) 在 C++ 中，若 $a = 3$、$b = 0$，則執行 `if(b != 0 && a / b > 1)` 不會發生除以零的錯誤。

(d) 在 C++ 中，任何 `for` 迴圈都可以被改寫成效果一模一樣的 `while` 迴圈。

(e) 在 C++ 中，若某個區域變數（local variable）與某個全域變數（global variable）同名，則在該區域變數的有效範圍（scope）內直接使用該名稱，存取到的是區域變數。

(f) 二分搜尋法（binary search）可以被用在未排序的單維陣列上。

(g) 在 C++ 中，若函數的參數是以傳值呼叫（call by value）的方式傳入，則在函數內修改該參數的值，不會影響呼叫端原本的變數。

(h) 在 C++ 中，一個函數內可以有多個 `return` 敘述，但每次呼叫該函數時最多只會執行到其中一個。

(i) 在 C++ 中，編譯和執行 `char str[5] = "hello";` 沒有出錯的機會。

(j) 在 C++ 中，若每次執行程式時都以相同的種子（seed）呼叫 `srand()` 一次，則該次程式執行的後續數次呼叫 `rand()` 產生的數列都會相同。

第二題

（20 分；每小題 2 分）請寫出下列 C++ 程式中每一個 cout 敘述的輸出結果，並在作答時將每個輸出結果對應到 cout 後的註解內的英文字母標記（如 (a)、(b) 等等）。若結果無法預測，請寫「unpredictable」。請假設所需的函式庫和 namespace 都已經被引入了，且程式使用 ASCII 字元編碼。

```
void g(int& x, int y) {
    x += y;
    y += x;
}

int main() {
    char c = 'A';
    cout << c + 2 << "\n"; // (a)
    cout << static_cast<char>(c + 2) << "\n"; // (b)
    int a[5] = {1, 2, 3}, b[5];
    cout << a[3] << " " << b[2] << "\n"; // first (c) and then (d)
    int* p = a;
    cout << *(p + 2) << "\n"; // (e)

    int total = 0;
    for(int i = 1; i <= 10; i += 3)
        total += i;
    cout << total << "\n"; // (f)

    int n = 41;
    int steps = 0;
    while(n > 0)
        if(n % 3 == 0)
            n /= 3;
        else
            n -= 2;
        steps++;
    cout << steps << "\n"; // (g)
    cout << n << "\n"; // (h)

    int x = 1, y = 2;
    g(x, y);
    cout << x << "\n"; // (i)
    cout << y << "\n"; // (j)

    return 0;
}
```

第三題

（35 分；每小題 5 分）請簡答下列問題。答案需精確，但不必太長。為了避免大家煩惱要敘述得多細，每小題都限 200 字，請大家挑重點寫即可。

(a) 在 C++ 中，當我們把一個靜態配置的陣列傳入函數時（例如 `void f(int a[])`），為何通常需要另外傳入一個代表陣列長度的參數？為何不能在函數內用 `sizeof` 求出長度？

(b) 在 C++ 中，我們提到過一些不建議盲目使用遞迴（recursion）的理由，也提過一些適合使用遞迴的場景。請寫出一個一般情況下使用遞迴的壞處，以及一個適合使用遞迴的場景。你只要寫出壞處和適合使用的場景即可，不用寫原因。

(c) 考慮以下遞迴函數

```
int weird(int n)
{
    if(n == 0)
        return 2;
    else if(n == 1)
        return 1;
    else
        return 3 * weird(n - 1) - weird(n - 2);
}
```

請問 `weird(5)` 的回傳值為何？

(d) 承上題，假設 $n \geq 0$，請撰寫一個迴圈版本的 C++ 函數

```
int weirdRep(int n);
```

來得到和 `weird()` 一模一樣的結果。

(e) 給定一個長度為 n 的未排序陣列，我們要查詢 k 個值是否存在於陣列中。方法一是對每個查詢值都做一次線性搜尋（linear search），整體時間複雜度為 $O(kn)$。方法二是先用一個 $O(n \log n)$ 的排序演算法將陣列排序，再對每個查詢值做二分搜尋。請問方法二的整體時間複雜度為何？當 k 大約滿足什麼條件時，方法二會比方法一有效率？（當然還有方法三，就是排序後做一次線性搜尋，但本題我們不考慮這個方法。）

(f) 在 C++ 中，考慮以下程式片段：

```
int* ptr = new int[10];
int* ptr2 = ptr;
delete [] ptr;
ptr2[3] = 7;
```

執行最後一行時會發生什麼問題？為什麼這種問題特別危險、難以除錯？

(g) 在 C++ 中，為何標準函式庫 `<climits>` 要提供 `INT_MAX` 這樣的常數，讓我們在自己的程式中使用，而不建議我們直接在程式中寫 `2147483647`？

第四題

（20 分；每小題 10 分）請為以下各小題撰寫一個虛擬碼（pseudocode），並為其推導時間複雜度並且以 Big-O 的形式呈現。你的虛擬碼只要是對的（可以得到正確答案、每個步驟都明確地可被執行），就能得到基本分數；如果它是對的但時間複雜度過高，則會被酌予扣分。如果你不會寫虛擬碼，也可以寫 C++ 程式。

(a) 寫一個函數，收到兩個只包含小寫英文字元的字串 s 和 t 後，計算若 s 中的每個字元最多只能使用一次，最多可以從 s 中挑出字元、重新排列後組出幾份完整的 t，並回傳此份數。例如 $s =$ `"aabbcca"`、$t =$ `"abc"` 時應回傳 2；$s =$ `"abc"`、$t =$ `"aab"` 時應回傳 0。t 中的字元可能重複，且 s 和 t 的字串長度各為 n 和 m，$m \geq 1$。

(b) 寫一個函數，收到一個無向圖的相鄰矩陣（adjacency matrix）A 和一個整數 v 後，計算與編號為 v 的節點「距離恰好為二」的節點個數。所謂距離恰好為二，是指該節點不是 v 本身、與 v 之間沒有直接相連的邊，但可以經由某一個節點以兩步從 v 抵達。A 是一個 $n \times n$ 的方陣，節點編號為 1、2 直到 n，$A_{ij} = A_{ji} = 1$ 表示節點 i 和 j 之間有一條邊，$A_{ij} = 0$ 則否。已知 $1 \leq v \leq n$、$A_{11} = A_{22} = \cdots = A_{nn} = 0$。

第五題

（10 分；每小題 5 分）你要用 C++ 撰寫一個可處理分子非負且分母為正的分數的程式。已知分數（fraction）一般表示為 $\frac{p}{q}$，其中 p 為非負整數、q 為正整數。可對分數進行的兩種運算如下：

- 約分：將 p 與 q 同除以兩者的最大公因數，例如 $\frac{6}{8}$ 約分後為 $\frac{3}{4}$。
- 回傳商數：若 $p \div q$ 的結果為 s 餘 r，則 s 即為 $\frac{p}{q}$ 的商數（整數部分），此時應回傳 s。

你的程式需要處理的分子與分母都在一個 `int` 變數能儲存的範圍，運算過程中不會發生溢位，且不考慮浮點數精確度問題。

(a) 根據上述定義，請用 C++ 實作一個完整的 `struct Fraction` 來表現分數。請明確定義成員變數（member variables）與成員函式（member functions）。對於成員函式，需定義函式標頭（header）與實作。你的答案將被依正確性、可讀性與易用性計分。當一個分數被約分，該分數本身的值應該發生變化，而該函數沒有回傳值；當一個分數的商數要被回傳時，該分數本身的值應該不發生變化，而該函數回傳該分數的商數值（`int`）。

(b) 用 C++ 實作一個可以回傳兩個分數相加後約分到不能再被約分後的分數的分子。給定兩個分數 $\frac{a}{b}$ 和 $\frac{c}{d}$，它們的和被定義為

$$\frac{a}{b} + \frac{c}{d} = \frac{ad + bc}{bd}。$$

請將 $\frac{ad+bc}{bd}$ 約分後回傳其分子。請決定要將此函式設為成員函數還是全域函數，並簡要地說明原因。